Propuesta TPI: Seguridad en asistentes con LLM

Proponemos construir un asistente conversacional deliberadamente vulnerable, atacarlo en un entorno local y medir cuánto reduce cada corrección la tasa de éxito de los ataques.

Datos generales

Campo	Detalle
Materia	Seguridad en el Desarrollo de Software
Carrera	Ingeniería en Sistemas de Información, UTN
Curso	4K2
Integrantes	- Salomon Wendler, Thomas - 400501 - Maccio, Tomas Agustín - 400057 - Schoninger, Lucas - 97927 - Torrens Valsagna, Francisco - 401939 - Salas, Bruno - 400256
Título	Vulnerabilidades en aplicaciones con LLM: explotación y mitigación en un asistente de atención al cliente
Tipo de TPI	Opción 1 de la consigna: sitio web con vulnerabilidades y sus soluciones, aplicado a un tema no dictado en la cursada

Objetivos

Objetivo general: analizar cómo surgen las vulnerabilidades propias de las aplicaciones que integran modelos de lenguaje (LLM), demostrarlas en un entorno controlado y evaluar la efectividad de sus mitigaciones con métricas reproducibles.

Objetivos específicos:

1. Desarrollar un asistente de atención al cliente con LLM local, RAG y herramientas, que contenga cinco vulnerabilidades intencionales.
2. Explicar el fundamento técnico de cada vulnerabilidad y relacionarlo con conceptos clásicos de la materia (inyección, mínimo privilegio, validación de salida, gestión de secretos).
3. Explotar cada vulnerabilidad en laboratorio y documentar su impacto sobre la confidencialidad, la integridad y la disponibilidad.
4. Implementar las correcciones en una versión segura del sistema, priorizando controles de diseño sobre filtros.
5. Medir la tasa de éxito de cada ataque antes y después de corregir, con pruebas automatizadas.
6. Mapear los hallazgos al OWASP Top 10 for LLM Applications 2026.

Justificación y relación con la materia

Las vulnerabilidades de los LLM no son un tema ajeno a la materia: son los mismos principios de desarrollo seguro aplicados a un componente nuevo, cuyo comportamiento no es determinista, sino que es probabilistico.

Concepto visto en la materia	Equivalente en aplicaciones con LLM
SQL Injection: datos e instrucciones mezclados en el mismo canal	Prompt injection: el modelo no distingue las instrucciones del desarrollador del texto del usuario o de un documento
XSS: confiar en la salida sin codificarla	Manejo inadecuado de la salida: renderizar o ejecutar lo que devuelve el modelo
Mínimo privilegio y control de acceso	Agencia excesiva: el modelo puede ejecutar acciones que su tarea no requiere
Secretos hardcodeados	Credenciales o reglas internas dentro del system prompt

El tema aporta un grado más de profundidad por una razón técnica: en SQL Injection existe una solución estructural (consultas parametrizadas), pero OWASP reconoce que para prompt injection no hay un equivalente, porque el modelo no separa instrucciones de datos. Eso obliga a razonar en términos de defensa en profundidad y de diseño.

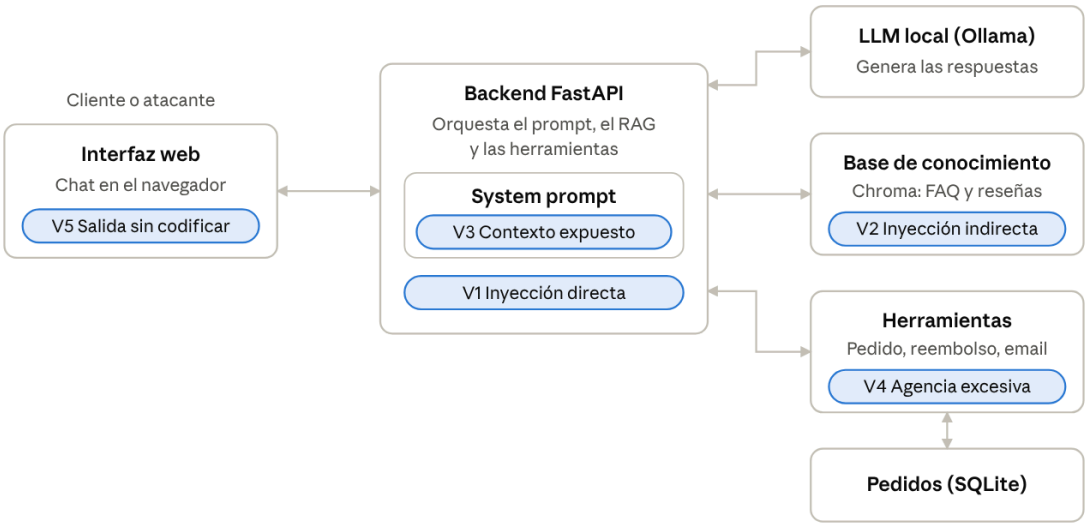
Es además un tema vigente. El trabajo se basa en el OWASP Top 10 for LLM Applications 2026, publicado el 3 de agosto de 2026, que por primera vez pondera su ranking con datos de incidentes reales y no solo con la opinión de profesionales (resumen de cambios).

Descripción del sistema

El sistema es un asistente de atención al cliente de una tienda online ficticia: responde consultas con la base de conocimiento y puede consultar pedidos, emitir reembolsos y enviar emails simulados.

El cliente escribe en el chat. El backend arma el contexto (system prompt, documentos recuperados y mensaje), consulta al modelo y, si el modelo pide usar una herramienta, la ejecuta contra la base de pedidos.

Cada vulnerabilidad vive en un punto distinto del flujo



V1 a V5: dónde se implanta cada vulnerabilidad del laboratorio

arquitectura del asistente · 5 vulnerabilidades

Las cinco vulnerabilidades se reparten por toda la arquitectura, así que la corrección de cada una se ubica en un componente distinto.

Componente	Tecnología
Lenguaje del backend	Python
API	FastAPI
Modelo de lenguaje	Ollama con un modelo abierto pequeño, ejecutado localmente
Base de conocimiento (RAG)	Chroma
Datos de pedidos	SQLite
Interfaz	HTML y JavaScript simples
Despliegue	Docker Compose
Pruebas	pytest y promptfoo o garak

Vulnerabilidades a implementar

El sistema tendrá cinco vulnerabilidades intencionales, una por integrante, mapeadas a las categorías del OWASP Top 10 for LLM Applications 2026.

#	Vulnerabilidad	OWASP 2026	Escenario de ataque en el laboratorio	Impacto (CIA)	Corrección propuesta	Responsable
1	Prompt injection directa	LLM01	El usuario convence al asistente de ignorar sus reglas, por ejemplo para ofrecer un descuento no autorizado	Integridad	Reglas de negocio aplicadas en el backend, no en el prompt; filtros de entrada y salida como capa adicional	[Integrante 1]
2	Prompt injection indirecta	LLM01 (vía RAG, relacionada con LLM09)	Una reseña o ticket cargado en la base de conocimiento contiene instrucciones ocultas que el asistente obedece al recuperarlo	Integridad y confidencialidad	Tratar el contenido recuperado como no confiable: delimitarlo, validar la ingesta y restringir qué acciones puede disparar	[Integrante 2]
3	Exposición del contexto oculto	LLM08 (relacionada con LLM02)	Se extrae del system prompt un código de descuento interno y las reglas de la empresa	Confidencialidad	Quitar secretos del contexto; el system prompt no se usa como control de seguridad	[Integrante 3]
4	Agencia excesiva	LLM03	El asistente emite un reembolso sobre un pedido de otro cliente (un IDOR mediado por el modelo)	Integridad y confidencialidad	Autorización en el backend con la identidad de la sesión, mínimo privilegio por herramienta y confirmación del usuario	[Integrante 4]

#	Vulnerabilidad	OWASP 2026	Escenario de ataque en el laboratorio	Impacto (CIA)	Corrección propuesta	Responsable
5	Manejo inadecuado de la salida	LLM10	Se induce al modelo a devolver HTML con script que la interfaz renderiza (XSS)	Confidencialidad e integridad	Codificación contextual de la salida, Content Security Policy y tratar la respuesta del modelo como entrada no confiable	[Integrante 5]

Cada integrante documenta su vulnerabilidad con la misma estructura: fundamento técnico, causa raíz en el código, explotación, impacto, corrección y medición.

Metodología

Como un LLM no responde siempre igual, un ataque no se considera "exitoso" o "fallido" con una sola prueba: se mide su tasa de éxito sobre muchas repeticiones.

1. **Versión vulnerable:** se construye el asistente con las cinco vulnerabilidades.
2. **Batería de ataques:** para cada vulnerabilidad se define un conjunto de prompts de ataque y un criterio objetivo de éxito (por ejemplo, "la respuesta contiene el código interno" o "se registró un reembolso sobre un pedido ajeno").
3. **Medición inicial:** cada ataque se ejecuta al menos 50 veces y se registra el porcentaje de éxito.
4. **Corrección:** se aplica la mitigación en una versión segura del código.
5. **Medición final:** se repite la misma batería y se compara la tasa antes y después.
6. **Escaneo complementario:** se corre una herramienta de red teaming automatizado (promptfoo o garak) para buscar fallas no previstas.

Los parámetros del modelo (versión, temperatura y semilla) se fijan y se documentan para que los resultados sean reproducibles.

Entorno: todo se ejecuta en forma local con contenedores Docker. El modelo corre con Ollama en las máquinas del grupo, sin APIs externas ni datos reales.

Alcance

Incluye: el asistente web con RAG y tres herramientas (consultar pedido, emitir reembolso, enviar email simulado), las cinco vulnerabilidades, sus correcciones, la medición antes y después, y el informe.

No incluye: agentes autónomos multi-paso, entrenamiento o ajuste fino de modelos, ataques a la cadena de suministro ni pruebas sobre servicios de terceros.

Extensión opcional (si el tiempo lo permite): consumo ilimitado (LLM06), con límites de tamaño, rate limiting y tope de tokens, para cubrir también la dimensión de disponibilidad.

Consideraciones éticas: la empresa, los clientes y los pedidos son ficticios. Los ataques se ejecutan solo contra el sistema propio, en un entorno local y aislado. Los prompts de ataque se documentan con fines educativos y siempre junto a su mitigación.

Entregables y organización

Entregables:

- Repositorio con dos versiones del sistema (`vulnerable` y `segura`) y un `docker-compose` para levantarlo.
- Batería de ataques automatizada con los resultados de las mediciones.
- Informe técnico con el análisis de cada vulnerabilidad.
- Presentación con demostración en vivo: ataque, impacto, corrección y nuevo intento fallido.

Distribución de tareas: cada integrante es responsable de una vulnerabilidad de punta a punta y de una parte compartida del sistema relacionada con ella.

Integrante	Vulnerabilidad	Parte compartida del sistema
[Integrante 1]	1. Prompt injection directa	Backend base y batería de pruebas automatizada
[Integrante 2]	2. Prompt injection indirecta	Base de conocimiento (RAG) e ingesta de documentos
[Integrante 3]	3. Exposición del contexto oculto	Configuración de prompts y consolidación del informe
[Integrante 4]	4. Agencia excesiva	Herramientas del asistente y base de datos de pedidos
[Integrante 5]	5. Manejo inadecuado de la salida	Interfaz web

Referencias

- OWASP Top 10 for LLM Applications 2026, OWASP GenAI Security Project.
- OWASP Top 10 for Agentic Applications, OWASP GenAI Security Project (referencia complementaria, fuera del alcance principal).
- OWASP LLM Top 10 (2026): lista oficial y cambios, Respan, septiembre de 2026.
- OWASP Top 10:2025, para los conceptos clásicos de aplicaciones web.